Exercise Sheet 09

The Living Ledger

House rules.

- First: `pip install -r requirements.txt` inside the lesson folder (venv active). That installs pandas and openpyxl (for Excel files).
- One file per exercise in `my_work/`, run from the lesson folder: `python my_work/ex1_open_the_file.py`.
- Every exercise starts with the same three lines (the **opener**). Type them, don't paste them:

```
import pandas as pd

pd.set_option("display.width", 120)
df = pd.read_csv("data/quests.csv")
```

- Printed tables must match the expected output exactly, including the numbers down the left (the **index**). If yours differ, you've filtered or sorted differently.
- Hand in: commit and push to `lesson-09`, then open a pull request. Tell Lars which Excel habit you miss, and which you don't.

Exercise 1 Open the ledger

5 min · `ex1_open_the_file.py`

> **Raven from Pell, the Quartermistress:** *"Orrin says pandas opens the whole quest ledger as a 'DataFrame'. Is that a spreadsheet? It sounds like a spreadsheet."*

(a) Print `type(df)`, `df.shape` and `list(df.columns)`.

(b) Print an empty line, then the first five rows of just these columns: quest_id, region, monster, difficulty, reward. Select columns with a list inside the brackets: `df[["a", "b"]]`.

(c) Print an empty line, then the last three rows of quest_id, monster, outcome (`.tail(3)`).

EXPECTED OUTPUT

```
<class 'pandas.DataFrame'>
(400, 12)
['quest_id', 'title', 'region', 'monster', 'difficulty', 'posted', 'days', 'reward',
'party_size', 'outcome', 'injuries', 'paid']

  quest_id          region  monster  difficulty  reward
0    Q0001       Thornwood  Goblins           1     260
1    Q0002       Saltmarsh  Bandits           4     550
2    Q0003       Thornwood  Bandits           2     620
3    Q0004  Northern Wilds  Bandits           2     670
4    Q0005  Northern Wilds  Goblins           1     410

    quest_id  monster  outcome
397    Q0398  Goblins     Fled
398    Q0399   Ghouls   Failed
399    Q0400  Goblins  Success
```

Hint: The numbers down the left are the **index**: row labels, starting at 0. They stick to their rows when you filter or sort.

Exercise 2 — One column at a time

10 min · `ex2_one_column.py`

> **Raven from Pell, the Quartermistress:** *"The reward column. Mean, median, max, total. And how many kinds of monster do we actually fight? The notice board says 8, the Scriptorium says 11. The Scriptorium counts 'goblins' and 'Goblins' separately."*

(a) `reward = df["reward"]`. Print its type and `reward.head(3)`.

(b) Print an empty line, then the mean (rounded to 2 dp), median, max and sum, each on its own line.

(c) Print an empty line, then how many different monsters there are (`.nunique()`) and the sorted list of them (`sorted(... .unique())`).

(d) Print an empty line, then `reward.describe().round(2)`.

EXPECTED OUTPUT

```
<class 'pandas.Series'>
0    260
1    550
2    620
Name: reward, dtype: int64

Mean: 587.72
Median: 470.0
Max: 4540
Total: 235090

Monsters: 8
['Bandits', 'Ghouls', 'Giant Rats', 'Giant Spiders', 'Goblins', 'Trolls', 'Wolves', 'Wyrm']

count     400.00
mean      587.72
std       481.11
min        90.00
25%       310.00
50%       470.00
75%       740.00
max      4540.00
Name: reward, dtype: float64
```

Hint: A single column is a **Series**. It behaves like last lesson's NumPy array, with an index and a name attached.

Exercise 3 — AutoFilter

10 min · `ex3_autofilter.py`

> **Raven from Captain Siv:** *"Every quest over 2,000 crowns. Every failure in the Northern Wilds. How many jobs in the Greyfen and the Saltmarsh, how many we didn't win, and how many at the old mill, because the miller says it's 'only the once'."*

(a) `big = df[df["reward"] > 2000]`. Print the count (`len(big)`) and the table of quest_id, region, monster, reward, outcome. Empty line.

(b) Failed in the Northern Wilds: region is Northern Wilds *and* outcome is Failed. Count, then quest_id, monster, difficulty, party_size, injuries. Empty line.

(c) Count quests whose region is Greyfen or Saltmarsh (`.isin([...])`), quests that were not a success (`!=`), and quests whose title contains "old mill" (`.str.contains(...)`).

EXPECTED OUTPUT

```
Over 2,000 cr: 7
    quest_id          region monster  reward outcome
14     Q0015  Northern Wilds    Wyrm    4540  Failed
24     Q0025       Thornwood    Wyrm    2420    Fled
48     Q0049       Saltmarsh    Wyrm    2960    Fled
162    Q0163       Ironhills    Wyrm    3420  Failed
164    Q0165         Greyfen    Wyrm    2510    Fled
340    Q0341       Thornwood    Wyrm    2530  Failed
345    Q0346       Ironhills    Wyrm    3530  Failed

Failed in the Northern Wilds: 9
    quest_id        monster  difficulty  party_size  injuries
14     Q0015           Wyrm           5           5         2
110    Q0111     Giant Rats           2           3         0
124    Q0125         Wolves           3           3         2
261    Q0262         Wolves           3           3         0
285    Q0286         Trolls           5           6         2
320    Q0321         Ghouls           3           3         0
343    Q0344  Giant Spiders           5           5         2
372    Q0373         Trolls           5           5         1
398    Q0399         Ghouls           3           3         1

Greyfen or Saltmarsh: 150
Not a success: 141
Anything at the old mill: 40
```

Hint: Same as NumPy: a comparison gives a mask, the mask goes in square brackets. Combine with `&` and `|`, brackets round each part.

Exercise 4 New columns

10 min · `ex4_new_columns.py`

> **Raven from Pell, the Quartermistress:** *"I always add the same helper columns: crowns per day, crowns per hero, one loot share, won or not, hard or not. Every. Single. Moon."*

(a) Add columns: `per_day` (paid / days, 2 dp), `per_hero` (paid / party_size, 2 dp), `share` (paid / (party_size + 1), 2 dp: the leader takes two shares), `won` (outcome is Success), `hard` (difficulty 4 or more) and `job` (monster + " in " + region).

(b) Print the shape, then the first five rows of quest_id, job, per_day, per_hero, share, won, hard. Empty line.

(c) Print the share of quests won (mean of the True/False column, 2 dp), the best crowns per quest-day, and the total crowns paid out.

EXPECTED OUTPUT

```
(400, 18)
  quest_id                       job  per_day  per_hero   share   won   hard
0    Q0001       Goblins in Thornwood    65.00     260.0  130.00  True  False
1    Q0002       Bandits in Saltmarsh    78.57     137.5  110.00  True   True
2    Q0003       Bandits in Thornwood   124.00     310.0  206.67  True  False
3    Q0004  Bandits in Northern Wilds    83.75     335.0  223.33  True  False
4    Q0005  Goblins in Northern Wilds    51.25     410.0  205.00  True  False

Share of quests won: 0.65
Best crowns per quest-day: 221.67
Total crowns paid out: 146150
```

Hint: `df["new"] = ...` creates the column; the right-hand side is column maths like last lesson. Round the whole column: `(df["a"] / df["b"]).round(2)`. Text columns join with `+`, row by row.

Exercise 5 Sorting

10 min · `ex5_sorting.py`

> **Decree from Queen Aldara:** *"Which quests pay best per day? Which are the worst ones we still won? Can I have it by region? And who has been in the guild longest? (Four questions, I know. I am the Queen.)"*

Add the `per_day` column from Exercise 4, and set `cols = ["quest_id", "region", "monster", "per_day"]`.

(a) "Top 5 crowns per quest-day:" then the five highest (`sort_values("per_day", ascending=False)[cols].head()`). Empty line.

(b) "Worst 3 quests that were actually won:" filter the successes first, then the three lowest per_day. Empty line.

(c) "By region, then reward (highest first):" the first three rows sorted by region A–Z and within that by reward, highest first (`ascending=[True, False]`), columns quest_id, region, monster, reward. Empty line.

(d) "Longest in the guild:" read `data/heroes.csv` into `heroes` and print the three earliest `joined` dates (name, class, joined).

EXPECTED OUTPUT

```
Top 5 crowns per quest-day:
    quest_id     region monster  per_day
392    Q0393  Thornwood  Trolls   221.67
121    Q0122  Thornwood  Trolls   197.14
213    Q0214  Thornwood  Trolls   185.71
369    Q0370  Ironhills  Trolls   178.89
211    Q0212  Ironhills  Trolls   174.44

Worst 3 quests that were actually won:
    quest_id          region     monster  per_day
342    Q0343         Greyfen  Giant Rats    14.29
181    Q0182         Greyfen  Giant Rats    15.00
227    Q0228  Northern Wilds  Giant Rats    15.45

By region, then reward (highest first):
    quest_id   region monster  reward
164    Q0165  Greyfen    Wyrm    2510
134    Q0135  Greyfen  Trolls    1180
371    Q0372  Greyfen  Trolls    1140

Longest in the guild:
                   name    class      joined
41   Kellan Fallowfield   Ranger  2012-01-14
71     Wren Nightingale   Cleric  2012-02-07
110   Doran Fallowfield  Warrior  2012-02-10
```

Hint: Dates in the format YYYY-MM-DD sort correctly even as text. That's why the Scriptorium uses it. (That, and Orrin insisted.)

Exercise 6 Excel in, CSV out

10 min · `ex6_excel_in_csv_out.py`

> **Raven from Orrin of the Clockworks:** *"The old ledger is a workbook, `data/guild_ledger.xlsx`, two sheets. Read it. Then write me a CSV of the quests where the party fled, so I can stop opening Excel at all."*

No opener this time: you're reading Excel instead of the CSV.

(a) `quests = pd.read_excel("data/guild_ledger.xlsx", sheet_name="Quests")`, and the same for the `"Notes"` sheet into `notes`. Print both shapes, then the last note (`notes["note"].tolist()[-1]`). Empty line.

(b) Filter the quests where the outcome is Fled, keep only quest_id, region, monster, difficulty, reward, sort by quest_id, and save with `.to_csv("my_work/fled_quests.csv", index=False)`.

(c) Read your new CSV back in and print its shape and first three rows, to prove it worked.

EXPECTED OUTPUT

```
(400, 12)
(5, 1)
The Grimoire of Dalvric reads this sheet. Do not rename it.

(82, 5)
  quest_id          region     monster  difficulty  reward
0    Q0006  Northern Wilds      Wolves           2     460
1    Q0007       Thornwood  Giant Rats           2     130
2    Q0014  Northern Wilds      Ghouls           3     950
```

Hint: `index=False` stops pandas writing the row numbers into the file as a column. Forget it once and you'll have a column called "Unnamed: 0" forever.

Exercise 7 Real dates STRETCH

15 min · `ex7_dates.py`

> **Raven from Pell, the Quartermistress:** *"Biscuit swears quests posted on a Friday go better. Biscuit also swears he didn't eat the candles. And the Queen wants the late-June quests, before Midsummer. From a column of text dates. Help."*

Opener, then `df["posted"] = pd.to_datetime(df["posted"])`.

(a) Add `month` (`.dt.month`) and `weekday` (`.dt.day_name()`). Print how many quests were posted in June, both years together.

(b) Filter the quests posted from 15 to 30 June 2025 (a real date column compares with `"2025-06-15"` just fine). Print the count and the table (quest_id, posted, weekday, monster, outcome). Empty line.

(c) Print the number of days between the first and the last quest in the ledger (`(max - min).days`). Empty line.

(d) Biscuit's theory: print how many quests were posted on a Friday, the share won among those, and the share won on any day (1 dp).

EXPECTED OUTPUT

```
Posted in June, both years: 33
Posted in the second half of June 2025: 8
    quest_id     posted    weekday        monster  outcome
293    Q0294 2025-06-17    Tuesday        Goblins   Failed
294    Q0295 2025-06-18  Wednesday  Giant Spiders  Success
295    Q0296 2025-06-20     Friday         Wolves  Success
296    Q0297 2025-06-20     Friday        Bandits  Success
297    Q0298 2025-06-24    Tuesday        Goblins  Success
298    Q0299 2025-06-25  Wednesday        Goblins  Success
299    Q0300 2025-06-27     Friday        Goblins  Success
300    Q0301 2025-06-29     Sunday        Bandits  Success

Days from the first quest to the last: 719

Posted on a Friday: 58
Won, if posted on a Friday: 70.7%
Won, any day: 64.8%
```

Hint: Once a column is a real date, `.dt.month`, `.dt.year` and `.dt.day_name()` all work. Text dates can't do any of that. Whether 70.7 % vs 64.8 % proves anything about Fridays is a question for lesson 10. Don't tell Biscuit.

Bug hunt Pages from the Grimoire, page 9

15 min · `bughunt/ironhills_bug.py`

Dalvric's last ever script: a quick word on the Ironhills for the Queen, sent by raven at two in the morning with the note "should work". **Three** bugs: one crashes (the error names what it can't find; read it very closely), one gives a wrong number (the Ironhills are not that busy), one is a sorting detail.

EXPECTED OUTPUT ONCE FIXED

```
Quests in the ledger: 400
Ironhills quests: 85
Ironhills crowns paid: 34730
Biggest reward in the Ironhills: Q0346 Wyrm 3530
Smallest reward in the Ironhills: Q0204 Giant Rats 140
```

One commit per bug, each message explaining what was wrong and why.

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.